(Task 1 of 12) This series of tutorials revolve around a central idea, **SMoL**, the **S**tandard **M**odel **o**f **L**anguages. This is the embodiment of the computational core of many of our widely-used programming languages, from C# and Java to JavaScript, and Python to Scala and Racket. All these languages (and many others), to a large extent, have a common computational core:

- lexical scope
- nested scope
- eager evaluation
- sequential evaluation (per "thread")
- mutable first-order variables
- mutable first-class structures (objects, vectors, etc.)
- higher-order functions that close over bindings
- automated memory management (e.g., garbage collection)

Over the course of the SMoL Tutor, you will become familiar with these language features through reading and evaluating programs. What goes in SMoL is, of course, a judgment call: when a feature isn't present across a large number of diverse languages (like static types), or shows too much variation between languages that have it (like objects), we do not include that in the *standard* model. But it is not a value judgment: the standard model is about what languages *are*, rather than what languages *should be*.

(Task 2 of 12) The Tutor often presents programs in two distinct languages as follows.

Pseudo [Run ▶] | Scala 3

```
print(1 + 1)    println(1 + 1)
```

You can switch the language of the program on the right by clicking the button in the top-right corner.

These programs should be equivalent unless otherwise noted, so you can answer Tutor questions based on any one of them. However, we recommend reading the code in at least two different languages to help generalize your understanding across multiple programming languages.

(Task 3 of 12) In this tutorial, we will learn about **definitions**.

The following program illustrates **variable definitions**.

Pseudo [Run ▶] | Scala 3

```
let x = 1       val x = 1
let y = 2       val y = 2
print(x)        println(x)
print(y)        println(y)
print(x + y)    println(x + y)
```

In this program, the first variable definition **binds** `x` to `1`, and the second variable definition binds `y` to `2`.

This program produces three values: the value of `x`, the value of `y`, and the value of `x + y`. These values are `1`, `2`, and `3`, respectively. In this Tutor, we will write the result of running this program on a single line as

```
1 2 3
```

rather than

```
1
2
3
```

Pseudo | 🎲

(Task 4 of 12) What is the result of running this program?

Pseudo [Run ▶] Python

```
let x = 1
let y = x + 2
print(x)
print(y)
```

```
x = 1
y = x + 2
print(x)
print(y)
```

```
1 3
```

4

5

You predicted the output correctly 🎉 🎉 🎉

The first definition binds `x` to the value `1`. The second definition binds `y` to the value of `x + 2`, which is `3`. So, the result of this program is `1 3`.

Click here to run this program in the Stacker.

Pseudo | 🎲

(Task 5 of 12) The following program illustrates **function definitions**.

Pseudo [Run ▶] Python

```
fun f(x, y):
  return (x + y) / 2
end
print(f(2 * 3, 4))
```

```
def f(x, y):
    return (x + y) / 2
print(f(2 * 3, 4))
```

This program produces `5`. It defines a function named `f`, which has two **formal parameters**, `x` and `y`, and **calls** the function with the **actual parameters** `6` and `4`.

Pseudo | 🎲

(Task 6 of 12) What is the result of running this program?

Pseudo [Run ▶] Python

```
fun f(x, y, z):
  return x + (y + z)
```

```
def f(x, y, z):
    return x + (y + z)
print(f(2, 1, 3))
```

```
end
print(f(2, 1, 3))
```

6

8

You predicted the output correctly 🎉 🎉 🎉

9

Click here to run this program in the Stacker.

Pseudo |

(Task 7 of 12) Function definitions can contain definitions, for example

Pseudo [Run ▶]

```
fun f(x, y):
  let n = x + y
  return n / 2
end
print(f(2 * 3, 4))
```

Lispy

```
(deffun (f x y)
  (defvar n (+ x y))
  (/ n 2))

(f (* 2 3) 4)
```

Pseudo |

(Task 8 of 12) What is the result of running this program?

Pseudo [Run ▶]

```
fun f(x, y, z):
  let p = y * z
  return x + p
end
print(f(2, 1, 3))
```

Python

```
def f(x, y, z):
    p = y * z
    return x + p
print(f(2, 1, 3))
```

6

12

The answer is 5.

▼ Textual explanation

13

Click here to run this program in the Stacker.

Pseudo |

What is the result of running this program?

Pseudo [Run ▶]

```
fun f(a, b, c):
  let q = b + c
  return a * q
end
print(f(3, 2, 1))
```

Scala 3

```
def f(a : Int, b : Int, c : Int) =
  val q = b + c
  a * q
println(f(3, 2, 1))
```

9

15

You predicted the output correctly 🎉 🎉 🎉

16

Pseudo | 🎲

(Task 9 of 12) We have two kinds of places where a definition might happen: the top-level **block** and function bodies (which are also **blocks**). A block is a sequence of definitions and expressions.

Blocks form a tree-like structure in a program. For example, we have four blocks in the following program:

Pseudo [Run ▶] JavaScript

```
let n = 42
fun f(x):
  let y = 1
  return x + y
end
fun g():
  fun h(m):
    return 2 * m
  end
  return f(h(3))
end
print(g())
```

```
let n = 42;
function f(x) {
  let y = 1;
  return x + y;
}
function g() {
  function h(m) {
    return 2 * m;
  }
  return f(h(3));
}
console.log(g());
```

The blocks are:

- the top-level block, where the definitions of `n`, `f`, and `g` appear
- the body of `f`, where the definition of `y` appears, which is a sub-block of the top-level block
- the body of `g`, where the definition of `h` appears, which is also a sub-block of the top-level block, and
- the body of `h`, where no local definition appears, which is a sub-block of the body of `g`

Any feedback regarding these statements? Feel free to skip this question.

18

(You skipped the question.)

19

Pseudo | 🎲

(Task 10 of 12) We use the term **values** to refer to the typical result computations. These include numbers, strings, booleans, etc. However, running a program can also produce an **error**.

For example, the result of the following program is `23 error` because you can't divide a number by `0`, so the program stops executing as soon as it runs into an error.

Pseudo [Run ▶] JavaScript

```
let x = 23
print(x)
print(x / 0)
print(x)
```

```
let x = 23;
console.log(x);
console.log(x / 0);
console.log(x);
```

Pseudo | 🎲

(Task 11 of 12) What is the result of running this program?

Pseudo [Run ▶] Python

```
let xyz = 42
print(abc)
```

```
xyz = 42
print(abc)
```

`error`

22

You predicted the output correctly 🎉 🎉 🎉

23

The variable `abc` is not defined.

Click here to run this program in the Stacker.

(Task 12 of 12) It is an error to evaluate an undefined variable.

24

Any feedback regarding these statements? Feel free to skip this question.

25

(You skipped the question.)

26

Pseudo | 🎲

Let's review what we have learned in this tutorial.

We have two kinds of places where a definition might happen: the top-level **block** and function bodies (which are also **blocks**). A block is a sequence of definitions and expressions.

Blocks form a tree-like structure in a program. For example, we have four blocks in the following program:

Pseudo [Run ▶] JavaScript

```
let n = 42
fun f(x):
  let y = 1
  return x + y
end
fun g():
  fun h(m):
    return 2 * m
  end
```

```
let n = 42;
function f(x) {
  let y = 1;
  return x + y;
}
function g() {
  function h(m) {
    return 2 * m;
  }
```

```
  return f(h(3))      return f(h(3));
end                 }
print(g())          console.log(g());
```

The blocks are:

- the top-level block, where the definitions of `n`, `f`, and `g` appear
- the body of `f`, where the definition of `y` appears, which is a sub-block of the top-level block
- the body of `g`, where the definition of `h` appears, which is also a sub-block of the top-level block, and
- the body of `h`, where no local definition appears, which is a sub-block of the body of `g`

It is an error to evaluate an undefined variable.

You have finished this tutorial 🎉 🎉 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1775699617746